

# hm-dianping 项目专题面试题库（校招 / 实习版，含参考回答）

说明：这份文档保留了专题题库的结构，并在每道题后补充了适合校招 / 实习场景的参考回答。回答风格尽量贴近“这是我自己做过的、能展开讲细节”的口径。

## 一、项目总览专题

### 1. 先用 1 分钟介绍一下你的 hm-dianping 项目，你主要负责了哪些部分？

参考回答：

我的 hm-dianping 是一个基于 Spring Boot + Redis + MySQL + Kafka + Nginx 的本地生活服务项目，业务上参考了大众点评和小红书的部分场景，主要包括用户登录、商户查询、探店博客、关注 Feed、优惠券和秒杀下单等功能。我负责的是后端整体开发和改造，不只是接口实现，还包括缓存体系、秒杀高并发链路、分布式锁、失败补偿、限流和测试补充这些工程化能力。如果面试官继续展开，我一般会重点讲商户缓存和秒杀链路，因为这两块最能体现我在后端设计和细节实现上的工作。

### 2. 这个项目和普通 CRUD 项目相比，最大的技术亮点是什么？

参考回答：

我觉得最大的区别是这个项目不只是增删改查，而是加入了比较完整的高并发和缓存设计。比如商户查询做了 `Caffeine + Redis` 两级缓存，秒杀链路做成了 `Redis Lua + Redis Stream + Kafka + MySQL` 的异步方案，还处理了一人一单、幂等、库存回滚和失败补偿。相比普通 CRUD，这个项目更能体现我对性能、并发和稳定性的考虑。

### 3. 你为什么会选这个项目作为实习 / 校招阶段的代表项目？

参考回答：

因为它业务不空，能讲清真实场景；技术也不浅，能覆盖 Redis、消息队列、缓存、高并发这些面试高频点。对校招来说，这种项目比单纯管理系统更能体现后端能力。另一方面，这个项目我自己后续做了不少增强改造，所以很多实现细节我能讲得比较扎实。

### 4. 这个项目里有哪些核心业务模块？它们之间大致是怎么关联的？

参考回答：

核心业务模块有用户模块、商户模块、博客模块、关注模块、优惠券模块和秒杀订单模块。用户登录之后可以查商户、发博客、关注别人、看 Feed，也可以参与优惠券秒杀。技术上这些模块共享同一套登录态、缓存、Redis 工具类和数据库访问层，秒杀模块还额外用到了 Lua、Redis Stream、Kafka 和补偿机制。

### 5. 如果面试官只让你讲一个最能体现你能力的模块，你会讲哪一个？为什么？

参考回答：

我会优先讲秒杀模块。因为它同时覆盖了 Redis、Lua、Kafka、MySQL、分布式锁、幂等、异常补偿这些后端高频点，技术密度最高，也最能体现我不是只会写接口。另外如果面试官更偏缓存，我也可以切到商户缓存这块，因为我做过两级缓存和失效同步的改造。

## 6. 这个项目是你从零搭起来的，还是在现有基础上做增强？你做了哪些改造？

参考回答：

项目基础业务框架是现有的，但后端很多关键能力我是按自己的理解做增强和改造的。比如秒杀链路从简单异步方案逐步改成了 `Redis Stream + Kafka` 的链路；缓存从单 Redis 缓存改成了两级缓存；另外还补了限流、失败补偿、配置脱敏和一些回归测试。可以理解成我是在已有业务骨架上，把它往更工程化的方向做了升级。

## 7. 你在项目里改造过哪些原有方案？为什么觉得原方案不够好？

参考回答：

我改造过的重点有两块。第一块是缓存，原来更多是单 Redis 缓存，查询热点数据时 Redis 压力会更集中，所以我补了本地缓存和失效同步。第二块是秒杀链路，原来链路能跑，但在幂等、失败补偿和异步削峰上不够完整，所以我把它做成了更清晰的消息流转和异常处理方案。

## 8. 这个项目的技术栈为什么选了 `Spring Boot + Redis + MySQL + Kafka`？

参考回答：

Spring Boot 适合快速搭后端服务，MySQL 负责业务数据持久化，Redis 主要解决缓存、会话、地理位置和高并发预处理的问题，Kafka 主要用来做秒杀订单的异步削峰和可靠消费。这套技术栈覆盖了校招后端最常见的核心能力，也比较适合讲清楚从业务到工程的设计过程。

## 9. 如果不看代码，只从业务角度讲，用户一次秒杀下单会经过哪些步骤？

参考回答：

用户点击秒杀之后，请求先到后端接口；后端会先通过 Redis 和 Lua 脚本快速判断库存够不够、用户是不是重复下单，并在 Redis 里先做预扣；如果通过，就把订单信息写进消息流，接口先快速返回；后面由消息消费端异步落库，真正扣数据库库存并创建订单；如果落库失败，再走补偿或回滚逻辑。这样设计是为了让接口快返回，同时把高并发压力从数据库前移到 Redis 和消息队列。

## 10. 你自己觉得这个项目里最难讲清楚的一段链路是哪一段？为什么？

参考回答：

我觉得最难讲的是秒杀链路。因为它不是单点功能，而是 Redis、Lua、Stream、Kafka、数据库、分布式锁和补偿逻辑串起来的一整条链路。面试时如果不按顺序讲，很容易说乱，所以我一般会先讲“为什么要这么设计”，再按“资格校验、异步削峰、数据库兜底、失败补偿”这个顺序展开。

---

## 二、登录与用户模块专题

---

## 1. 你的登录功能是怎么做的？为什么选择验证码登录而不是账号密码登录？

参考回答：

登录是手机号验证码登录，流程是先发送验证码到 Redis，再由用户提交手机号和验证码进行校验，校验通过后生成 token，并把用户简要信息写到 Redis 中。选择验证码登录一方面是因为这种项目更贴近移动端业务场景，另一方面它能自然引出 Redis 的使用，比如验证码过期、登录态刷新、拦截器校验这些点。

## 2. 验证码为什么放 Redis，不直接放数据库？

参考回答：

验证码本质上是一个短期有效的临时数据，生命周期很短，而且读写频率高，不适合落数据库。放 Redis 可以直接设置 TTL，到期自动失效，读写也更快。相比数据库，这种场景 Redis 更轻量、更合适。

## 3. 你的验证码 key 是怎么设计的？过期时间为什么这么定？

参考回答：

验证码 key 设计成 `login:code:{phone}`，这样按手机号维度隔离，查的时候也很直接。过期时间我设置得比较短，一般是 2 分钟左右，因为验证码本身就是高时效数据，时间太长不安全，太短又可能影响用户体验，所以 2 分钟是比较常见的折中。

## 4. 用户登录成功后，为什么把用户信息放 Redis，而不是直接放 Session？

参考回答：

因为 Redis 更适合做分布式登录态共享，如果后面服务做多实例部署，不同节点都能读到同一份登录信息。传统 Session 更适合单体或需要粘性会话的场景，而 Redis 存 token 更容易扩展，也更常见。对我这个项目来说，用 Redis 登录态和整体技术方案更统一。

## 5. 你项目里的 Token 是怎么生成和校验的？

参考回答：

登录成功后，我会生成一个随机 token，一般可以理解成 UUID 风格的值，然后把用户信息以 Hash 或类似结构存到 `login:token:{token}` 对应的 Redis key 里。后续请求带上 token，拦截器就去 Redis 查这条登录态是否存在，存在就说明登录有效。这个方案本质上是“token 做索引，Redis 存会话数据”。

## 6. 为什么要设计 `login:token:{token}` 这种 key？

参考回答：

这样设计主要是为了分层清晰和方便管理。前缀 `login:token:` 能直接表示这是登录态相关数据，后面拼 token 能保证唯一性，也方便后面做批量清理和排查。Redis 里 key 有明确业务前缀是一个很常见的习惯。

## 7. 登录拦截器和刷新 Token 的拦截器分别做了什么？顺序为什么不能乱？

参考回答：

刷新 token 的拦截器主要负责从请求里拿 token，如果 Redis 里存在这条登录态，就把用户信息刷新到当前线程上下文，并顺便续期。登录拦截器则负责判断当前接口是否要求登录，如果要求登录但上下文里没有用户，就拦截掉。顺序不能乱，因为必须先尝试解

析和刷新 token，后面登录校验才能知道当前用户到底有没有登录。

## 8. 你为什么要用 ThreadLocal 保存用户信息？它解决了什么问题？

参考回答：

因为用户信息一旦在拦截器阶段拿到了，后面的 controller、service 都可能要用。如果每一层都手动传 userId，代码会很啰嗦，所以我用 ThreadLocal 把当前请求的用户信息放到线程上下文里，后面直接取就行。它解决的是“请求内用户上下文透传”的问题。

## 9. ThreadLocal 在请求结束后如果不清理，会有什么问题？

参考回答：

最大的风险是线程复用导致脏数据串请求，尤其在线程池环境里更明显。比如上一个请求的用户信息残留到下一个请求里，就可能带来严重的权限问题。所以请求结束后一定要清理 ThreadLocal，这是使用它时必须注意的点。

## 10. 如果面试官问你这个登录方案和 JWT 的区别，你怎么回答？

参考回答：

我这个方案更偏“Redis 会话型 token”，核心是 token 只是一个凭证，真正的用户状态在 Redis 里。JWT 则是把用户信息和签名放在 token 自身里，服务端可以不查库也不查 Redis 就完成校验。前者的优点是可以主动失效、续期灵活，后者的优点是更无状态，但要做强制下线 and 续期会相对麻烦一些。

---

## 三、Redis 与缓存专题

---

### 1. 你项目里哪些地方用了 Redis?

参考回答:

Redis 在项目里用得比较多，主要包括验证码、登录态、商户缓存、商户 GEO、博客点赞、关注集合、Feed 流、签到、秒杀库存、一人一单集合，还有消息流转入口。也就是说它既承担了缓存作用，也承担了一部分高并发场景下的业务状态处理。

### 2. 商户查询为什么适合用缓存?

参考回答:

因为商户详情和商户列表属于读多写少的数据，用户访问非常频繁，但更新频率没有那么多高。这种典型的高频读场景特别适合缓存。把热点查询前移到 Redis 或本地缓存，能明显减少数据库压力。

### 3. 你缓存商户信息时，Redis 的 key 是怎么设计的?

参考回答:

商户详情一般设计成 `cache:shop:{id}`，这样业务含义很清晰，也方便按 id 精确失效。商户类型列表则是固定 key，比如 `cache:shop-type:list`。我比较重视 key 命名，因为后面排查线上问题时，规范的 key 会省很多时间。

### 4. 什么是缓存穿透？你在项目里是怎么解决的？

参考回答：

缓存穿透就是查询一个数据库里根本不存在的数据库，请求每次都打不到缓存，最终全落到数据库。我的处理方式是空值缓存，也就是数据库查不到时，在 Redis 里也写一个短 TTL 的空标记。这样下一次同样的请求就不会再打到数据库。

## 5. 什么是缓存击穿？你在项目里用了哪两种思路？

参考回答：

缓存击穿一般是某个热点 key 过期瞬间，大量请求同时打到数据库。我在项目里用了两种常见思路：一种是互斥锁，也就是只允许一个线程去查库并重建缓存；另一种是逻辑过期，也就是先返回旧值，同时后台异步重建缓存。两种方案我都保留了，方便根据场景选择。

## 6. 什么是缓存雪崩？你的项目里做了哪些预防？

参考回答：

缓存雪崩是大量 key 在同一时间失效，导致请求同时涌向数据库。我的项目里主要通过合理 TTL、空值缓存、热点数据本地缓存以及不把所有 key 设置成同一时刻过期这些方式做预防。如果是更大的线上系统，还可以进一步做随机过期时间或者多级缓存。

## 7. 为什么空值缓存能解决缓存穿透？它有没有副作用？

参考回答：

因为空值缓存把“不存在”这个结果也缓存起来了，后续同样的请求就不会再打数据库。副作用主要有两个，一个是会占一点 Redis 内存，另一个是如果这个数据后来真的新建了，短时间内可能还会读

到空标记。不过因为空值 TTL 通常比较短，所以这个副作用是可接受的。

## 8. 逻辑过期和互斥锁方案分别适合什么场景？

参考回答：

互斥锁更适合要求数据一致性更高、而且可以接受少量线程等待的场景。逻辑过期更适合热点数据，优点是大部分请求都能快速返回，不会一起阻塞，但短时间内可能返回旧数据。我的理解是，逻辑过期更偏“性能优先”，互斥锁更偏“数据新鲜度优先”。

## 9. 你后来为什么又加了本地缓存 Caffeine？

参考回答：

因为商户详情这类热点数据如果全部压在 Redis 上，Redis 还是会承受很高读取压力。加了 Caffeine 之后，同一个实例上的热点请求很多时候可以直接命中本地内存，不需要再走网络访问 Redis。这样可以进一步降低延迟，也减轻 Redis 压力。

## 10. 本地缓存和 Redis 同时存在时，会带来什么一致性问题？

参考回答：

最大的问题是某个节点更新了数据之后，Redis 虽然删掉了，但其他节点的本地缓存可能还是旧值。也就是说，分布式共享缓存和单机本地缓存之间会出现短暂不一致。这也是为什么我后面又补了失效通知机制。

## 11. 你的项目里本地缓存失效是怎么做的？

参考回答：

更新商户信息时，我会先更新数据库，再删 Redis，同时删掉当前节点本地缓存，然后通过 Redis Pub/Sub 发一条失效消息。其他节点订阅到这条消息后，再把各自本地缓存清掉。这样做是“主动失效 + TTL 兜底”的组合。

## 12. 为什么要引入 Redis Pub/Sub 来通知本地缓存失效？

参考回答：

因为本地缓存只存在于各个 JVM 里，不共享。如果不做广播，当前节点删了本地缓存，其他节点还是旧值。用 Pub/Sub 可以把“某个 key 失效了”这个事件广播出去，让所有节点都同步清理。

## 13. 如果 Pub/Sub 消息丢了，会发生什么？你是怎么兜底的？

参考回答：

如果消息丢了，最直接的结果就是某些节点短时间内可能继续读到旧的本地缓存。我的兜底方式是本地缓存本身也设置 TTL，即使广播失败，过一段时间缓存也会自动失效。也就是说，这块我接受的是“短暂最终一致”，而不是绝对强一致。

## 14. 你为什么给商户详情和商户类型分开配置本地缓存？

参考回答：

因为这两类数据的访问模式和更新频率不一样。商户详情 key 多、访问分散，但热点商户会非常热；商户类型列表 key 少、变化更少、适合更长缓存时间。分开配置之后，可以更细地控制容量和 TTL。

## 15. 本地缓存最大容量和过期时间你是怎么考虑的？

参考回答：

我主要是根据数据特点和访问频率来定。比如商户详情数量更多，但单条对象不大，所以我给它更大的容量；商户类型列表总数少，所以容量更小但 TTL 可以更长。对校招项目来说，这个值不是精确压测调出来的，但我会明确说明它是基于业务特征做的初步配置。

## 16. 如果面试官问你“为什么不全都用 Redis，不要 Caffeine”，你怎么答？

参考回答：

只用 Redis 当然也能做，但这样所有热点查询都要经过网络访问 Redis，延迟和压力都会更大。加 Caffeine 后，本机热点读可以直接走内存，会更快。代价是一致性处理更复杂，所以我后面才补了失效广播和 TTL 兜底。

## 17. Redis 热 key 会带来什么问题？你的项目里哪里可能产生热 key？

参考回答：

热 key 会导致某些 Redis key 被极高频访问，进而让单点压力过高，甚至影响整体延迟。项目里最容易出现热 key 的地方是热点商户详情、热门博客点赞数据、秒杀库存 key。越是用户集中访问的内容，越容易形成热 key。

## 18. 如果某个商户特别热门，你现在的缓存方案还能撑住吗？为什么？

参考回答：

比单 Redis 缓存方案更能撑住，因为我加了本地缓存，很多请求可以在 JVM 内部直接命中。不过如果热度再继续上升，还是可能要进一步做 TTL 优化、热点预热或者更细的监控。也就是说，现在这个方案对校招项目已经比较完整，但不是无限上限。

---

## 四、商户查询与 GEO 专题

---

### 1. 你的商户分页查询和按地理位置查询分别是怎么实现的？

参考回答：

普通商户查询是按类型去数据库分页查，如果是商户详情则优先走缓存。按地理位置查询时，我会先走 Redis GEO，根据坐标和半径筛出附近商户，再拿这些商户 id 去数据库查详情，并按 Redis 返回的顺序恢复距离排序。两种查询的核心区别是，一个以数据库分页为主，一个以 Redis GEO 为主。

### 2. 为什么普通分页查询不能直接满足“附近商户”场景？

参考回答：

因为“附近商户”本质上不是普通的条件过滤，而是带有地理距离排序的问题。数据库普通分页只能保证按页取数据，不能天然高效支持“距离我最近”的排序。所以这个场景更适合交给 Redis GEO 这种专门的数据结构处理。

### 3. 你为什么选 Redis GEO 做附近商户查询？

参考回答：

因为 Redis GEO 天然支持经纬度存储、附近搜索和距离计算，使用成本低，查询速度也比较快。对于我这个项目的体量来说，用 GEO 就能比较自然地实现“附近商户”功能，不需要自己额外设计复杂的地理索引。

## 4. GEO 查询完之后，为什么还要再查一次数据库？

参考回答：

因为 Redis GEO 里只适合存经纬度和商户 id，不适合把完整商户对象全塞进去。真正展示给前端时，还需要商户名称、地址、图片这些完整字段，所以要根据 GEO 返回的商户 id 再查数据库或者缓存，组装出完整结果。

## 5. 你为什么要保留商户 id 的顺序，再去数据库里 ORDER BY FIELD？

参考回答：

因为 GEO 返回的结果已经按距离排好了，如果我后面直接按 id 去数据库查，数据库默认不会保留这个距离顺序。为了保证最终结果还是“离我最近的在前面”，我需要把 Redis GEO 返回的 id 顺序带过去，再按这个顺序重新排序。

## 6. 如果 Redis GEO 查不到数据，你的代码是怎么兜底的？

参考回答：

如果查不到，我会回退到普通分页查询，至少保证接口还能返回结果，而不是直接空掉或者报错。这样做虽然不再是严格的“附近商户”，但能提升系统可用性。兜底逻辑更多是保证服务稳定。

## 7. 附近商户查询为什么要限制半径？半径设大了有什么问题？

参考回答：

限制半径一方面更符合“附近”的业务语义，另一方面也能避免一次查出太多数据。半径太大时，Redis GEO 返回结果会变多，后面的数据库查询和排序开销也会明显增加。所以一般都会给一个合理的搜索范围，比如几公里以内。

## 8. 如果面试官问你 GEO 的底层原理，你会怎么解释到校招合适的深度？

参考回答：

校招场景下我会说，Redis GEO 底层本质上还是借助有序集合来做的，只是对经纬度做了特殊编码，方便按地理位置做范围搜索和距离计算。我不会上来就讲特别底层的数学细节，而是重点讲它适合“附近搜索”这个业务场景。

## 9. 如果商户坐标变化了，你的 GEO 数据怎么更新？

参考回答：

如果商户坐标变了，就需要重新写 GEO 数据，相当于更新该商户在对应 GEO key 里的坐标。实际实现上可以理解成重新 `add` 一次同一个商户 id 的经纬度，同时把商户详情缓存一起失效掉，保证数据一致。

## 10. 你觉得这个模块还有哪些可以继续优化的点？

参考回答：

可以继续优化的点包括：增加更细粒度的缓存、热点区域预热、坐标更新和缓存更新的一致性处理，以及对 GEO 查询结果做分页优化。再往后如果体量更大，还可以考虑专门的搜索或地理位置服务。

---

## 五、博客、点赞、关注 Feed 专题

---

### 1. 你的博客点赞为什么用 Redis ZSet，而不是 Set？

参考回答：

因为点赞不只是要去重，有时还要保留顺序，比如展示最近点赞的前几个人。Set 只能表示“有没有点过赞”，而 ZSet 既能去重，又能通过 score 记录时间顺序。所以我选了 ZSet。

### 2. ZSet 里的 member 和 score 分别存了什么？

参考回答：

member 存的是用户 id，score 存的是点赞时间戳。这样我既能判断某个用户是否点过赞，也能按时间倒序拿到最近点赞列表。

### 3. 你为什么需要 score？如果没有 score 行不行？

参考回答：

如果只是做点赞去重，理论上 Set 也行。但我项目里还要支持“谁最近点过赞”这类展示，所以 score 很有意义。没有 score，就没法方便地做有序展示。

## 4. 你怎么判断当前用户有没有给某篇博客点过赞？

参考回答：

我会去 Redis 的点赞 ZSet 里查这个用户 id 的 score，如果 score 存在，就说明点过赞；如果不存在，就说明没点过。这个判断比每次去数据库查更快。

## 5. 你怎么拿到一篇博客最近点赞的前几个人？

参考回答：

我会从对应博客的点赞 ZSet 里按 score 排序取前几条，再根据用户 id 去查用户信息。因为 ZSet 本身就是有序的，所以这种“最近点赞列表”做起来很自然。

## 6. 点赞功能为什么要同时改数据库和 Redis？

参考回答：

数据库是最终持久化数据，Redis 是为了高效查询和展示。只改数据库会导致查询慢，只改 Redis 又不能保证持久化。所以我会数据库和 Redis 一起维护，不过数据库仍然是最终真实来源。

## 7. 如果数据库更新成功，但 Redis 更新失败了，会发生什么？

参考回答：

这样会导致短时间内点赞展示和真实数据不一致，比如数据库点赞数已经加了，但 Redis 里的用户点赞集合没更新。这个问题本质上是缓存和数据库双写一致性问题。对这种项目，我会接受短时不一致，并通过后续刷新或重新加载来恢复。

## 8. 关注功能为什么适合用 Redis Set?

参考回答:

因为关注关系天然是一个集合，而且最常见的操作是“是否关注”和“共同关注”。Set 的成员唯一、不重复，特别适合这个场景。比如 `SISMEMBER` 判关注、`SINTER` 求共同关注，都很方便。

## 9. 共同关注是怎么做出来的？为什么这个场景适合 Redis?

参考回答:

共同关注就是把当前用户和目标用户的关注集合做交集，得到双方都关注的人。Redis Set 直接支持交集运算，所以这个功能实现起来非常简单，而且效率也高。对这类关系型集合场景，Redis 很合适。

## 10. 你的 Feed 流是推模式、拉模式还是推拉结合？为什么？

参考回答:

我项目里更接近推模式，也就是用户关注了别人后，对方发博客时会把博客 id 推到粉丝的收件箱里。这样用户进首页时查自己的收件箱就行，读取会更快。缺点是写扩散更明显，但对这个项目体量是可以接受的。

## 11. 关注推送为什么用 ZSet?

参考回答:

因为 Feed 流需要按时间排序，ZSet 很适合做“按时间倒序查看动态流”。member 可以是博客 id，score 可以是发布时间戳，这样后面滚动分页就比较自然。

## 12. 滚动分页为什么不能直接用传统的 page/current?

参考回答：

因为 Feed 流这类按时间变化的数据，如果用普通页码分页，前面插入新数据后，后面的页内容很容易错乱、重复或遗漏。滚动分页更适合这种时间流场景，它是基于“上一次查到哪里”继续往后翻。

## 13. 你项目里的滚动分页具体用了哪几个参数？

参考回答：

一般会带上 `lastId` 或 `maxTime` 这类上次查询到的时间分值，再配合偏移量 `offset` 处理同分值数据。这样既能保证连续翻页，也能避免重复和漏数据。

## 14. 如果面试官问你“点赞和 Feed 这两个 Redis 结构为什么不同”，你怎么解释？

参考回答：

点赞更关注的是“某个用户有没有点过赞”和“最近谁点过赞”，所以用 ZSet 存用户和时间。Feed 则关注的是“当前用户按时间看到哪些内容”，本质上是另一种按时间排序的收件箱。虽然都用了 ZSet，但它们的业务含义和 member、score 的设计是不一样的。

---

## 六、秒杀链路专题

---

### 1. 你项目里秒杀链路完整流程是什么？请按顺序讲出来。

参考回答：

用户请求先进入秒杀接口，接口拿到用户 id 和优惠券 id 后，先执行 Redis Lua 脚本，脚本会判断库存是否充足、用户是否重复下单，并在 Redis 中先预扣库存、记录一人一单状态；如果通过，就把订单信息写入 Redis Stream；后面 Relay 从 Stream 里读取消息并转发到 Kafka；Kafka 消费者异步落库，真正扣减数据库库存并创建订单；如果消费失败，再根据异常类型决定重试、进入死信或者回滚 Redis 状态。这条链路的核心目标是“前端快返回、数据库削峰、最终一致”。

## **2. 为什么秒杀不能像普通下单一样，直接查库扣库存？**

参考回答：

因为秒杀场景并发太高，如果所有请求都直接打数据库，数据库很容易成为瓶颈，还可能出现锁竞争严重、响应很慢的问题。秒杀的核心是把资格校验和流量拦截尽量前移，让数据库只处理少量真正有效的请求。

## **3. 你为什么把资格校验放在 Redis Lua 里做？**

参考回答：

因为 Lua 能在 Redis 里把多步操作合成一个原子操作。秒杀里最关键的是“查库存、查是否重复下单、扣库存、记录状态、写消息”这些动作不能被并发打断，所以用 Lua 很合适。这样可以减少多次网络往返，也能保证一致性。

## **4. Lua 脚本在你的秒杀链路里到底做了哪几件事？**

参考回答：

它主要做了四件事：判断库存够不够；判断用户是不是已经下过单；如果通过，就预扣 Redis 库存；最后把订单消息写进 Redis Stream。也就是说，Lua 负责把秒杀资格判断和消息入队这块一次性做完。

## 5. 什么叫“一人一单”？你是在哪几层做这个保证的？

参考回答：

一人一单就是同一个用户对同一张秒杀券只能下一次单。我主要在三层做保证：Redis 层通过 Lua 和用户集合做第一层快速拦截；数据库层通过唯一索引做最终兜底；业务层还会结合分布式锁降低并发冲突概率。这样做是多层防线，而不是只依赖单点。

## 6. 为什么你做了 Redis 校验以后，数据库层还要再做一次兜底？

参考回答：

因为 Redis 预扣和数据库落库之间是有时间差的，而且异步链路里还有重试、重复消费这些因素。Redis 能解决大部分并发问题，但不能完全代替数据库最终约束。所以数据库唯一索引和库存条件仍然必须保留。

## 7. 秒杀时库存为什么先在 Redis 里预扣，而不是直接更新数据库？

参考回答：

因为 Redis 更适合承接高并发瞬时请求，预扣库存可以在前面快速过滤无效请求，避免每个请求都打数据库。数据库只负责最后的真实落库和强约束，这样整体吞吐量会更高。

## 8. Redis 预扣库存和数据库真实库存不一致时怎么办？

参考回答：

如果后面数据库落库失败，就需要做 Redis 回滚，把预扣掉的库存补回去，同时把一人一单集合里的用户状态清掉。也就是说，Redis 预扣不是最终结果，最终还是要靠数据库成功落库来确认。

## 9. 你的项目里 Redis 回滚库存是在哪些场景触发的？

参考回答：

主要是在消息消费端落库失败，而且这个失败被判断为不应该继续重试时触发，比如数据库库存不足、业务校验失败、唯一约束冲突之外的一些不可恢复异常。简单理解就是：Redis 先扣了，但后面订单没成，就要补回去。

## 10. 什么情况下你会认为这是“可恢复异常”，什么情况下是“不可恢复异常”？

参考回答：

像临时网络波动、数据库短暂不可用、消息系统短时异常，我会倾向认为是可恢复异常，可以重试。像业务数据本身有问题、库存已经没有、逻辑明确不可能成功的情况，我会认为是不可恢复异常，不应该盲目重试。这个区分是为了避免无效重试。

## 11. 为什么唯一索引也能帮助保证一人一单？

参考回答：

因为唯一索引从数据库层面规定了 `(user_id, voucher_id)` 组合不能重复。即使前面的 Redis 或消息链路有漏网请求，数据库插入时也会因为唯一索引冲突失败。它是最终兜底保障。

## 12. 你的数据库扣库存 SQL 为什么要带 `stock > 0`?

参考回答：

这是最基础的乐观约束，能避免库存已经是 0 了还继续往下扣。即使并发请求同时到数据库，只有满足 `stock > 0` 的更新才会成功，这样可以避免超卖。

## 13. 如果两个请求同时走到数据库扣库存，会不会超卖？

参考回答：

正常情况下不会，因为 SQL 本身带了 `stock > 0`，数据库只会让真正还有库存的更新成功。再加上前面 Redis 已经先拦掉一大部分请求，所以数据库层的并发压力和超卖风险都会小很多。

## 14. 你为什么还要再加 Redisson 分布式锁？

参考回答：

Redisson 锁不是唯一防线，但它能进一步减少同一用户并发创建订单时的竞争，特别是在消息消费和异步落库阶段。它更多是降低并发冲突概率，减轻数据库唯一索引冲突带来的无效操作。

## 15. 如果面试官问你“有唯一索引了，为什么还要锁”，你怎么答？

参考回答：

唯一索引是最终兜底，但它发生作用时已经走到数据库插入阶段了；锁则是在更前面减少同一用户的并发冲突，能少做很多无效工作。也就是说，一个是“最后挡住错误”，一个是“前面尽量不让错误发生”。

## **16. 秒杀成功时，接口为什么能很快返回？ 订单不是还没真正落库吗？**

参考回答：

因为我把真实落库放到了异步链路里。接口只要完成 Lua 校验、预扣库存和消息入队，就可以先返回订单 id 或成功结果，不需要同步等待数据库写入。这样就把高并发下最慢的数据库操作从用户请求线程里拆出去了。

## **17. 这种“先返回成功、后异步落库”的设计， 有什么风险？**

参考回答：

风险就是用户感知上的“成功”和数据库最终结果之间会有短暂差异。比如用户已经收到成功响应，但后面消息消费失败了，订单可能最终没创建成功。所以这种设计一定要配套补偿、幂等和失败处理机制，不能只追求接口快返回。

## **18. 你的项目里是怎么处理“用户看到成功， 但最终落库失败”这种情况的？**

参考回答：

我这边的思路是把失败分成可恢复和不可恢复两类。可恢复的就继续重试；不可恢复的就做 Redis 回滚，并记录失败任务或死信消息，方便后续补偿和排查。对前端来说，这类问题更适合通过订单状态、补偿处理或后续查询来感知最终结果。

## 19. 如果面试官让你描述秒杀链路里最核心的三道防线，你会怎么总结？

参考回答：

第一道是 Redis Lua，快速做库存和一人一单校验；第二道是异步消息链路，通过 Stream 和 Kafka 做削峰和解耦；第三道是数据库唯一索引、`stock > 0` 和分布式锁做最终一致性兜底。这样三层分别承担“前置拦截、异步缓冲、最终约束”的职责。

---

## 七、Kafka 与消息队列专题

---

### 1. 你的项目里为什么已经用了 Redis Stream，还要再引入 Kafka？

参考回答：

因为 Redis Stream 更像一个轻量级入口缓冲，而 Kafka 更适合做正式的高吞吐异步消息队列。我的项目里保留了 Stream 作为 Lua 脚本之后的第一站，再由 Relay 转发到 Kafka，这样既能减少改动成本，也能把异步消费能力提升起来。

### 2. Redis Stream 和 Kafka 在你的项目里分别承担什么职责？

参考回答：

Redis Stream 主要承担的是秒杀接口之后的快速入队缓冲，Kafka 承担的是正式的异步削峰、可靠消费、重试和死信处理。一个更靠近入口，一个更靠近正式消息系统。

### 3. 你项目里的 Relay 中转层是干什么的？

参考回答：

Relay 会从 Redis Stream 里读取订单消息，然后转发到 Kafka，对 Kafka 来说它是一个生产者。它相当于把“Lua 已经写入 Stream”的现有入口和“后端正式用 Kafka 消费”的新方案桥接起来。

## 4. 为什么不直接在业务代码里把消息发到 Kafka？

参考回答：

因为我这个项目的秒杀入口已经在 Lua 里完成了原子校验和 Stream 写入，如果直接改成同步发 Kafka，链路改动会更大，而且入口线程也会更依赖 Kafka 是否可用。加一个 Relay 可以让项目平滑演进，同时保留入口缓冲能力。

## 5. 你的 Kafka Topic 是什么？消息体里包含哪些关键字段？

参考回答：

核心 Topic 是秒杀订单相关的 topic，比如 `seckill-order`。消息体里至少会带订单 id、用户 id、优惠券 id，还会带创建时间和 `traceId` 这种方便追踪的字段。因为消费者落库时必须依赖这些关键信息。

## 6. 你为什么给消息加 `traceId`？它有什么用？

参考回答：

主要是为了方便排查链路问题。后面如果需要从 Redis Stream、Relay、Kafka、消费者日志一路往后追，就可以通过同一个 `traceId` 把一笔订单请求串起来。虽然这是校招项目，但这个思路更贴近真实工程实践。

## 7. 消费者为什么要手动 ACK，而不是自动提交？

参考回答：

因为自动提交有可能在业务还没真正处理成功前就把 offset 提交了，万一后面报错，就可能出现消息丢失语义。手动 ACK 可以做到“处理成功再确认”，更适合订单这类不能随便丢的业务。

## 8. 什么叫消息重复消费？你的项目里是怎么做幂等的？

参考回答：

重复消费就是同一条消息因为重试、网络抖动、消费者重启等原因，被处理了不止一次。我的幂等主要靠几层一起做：前面 Redis 一人一单校验、数据库唯一索引、消费端对重复插单的兜底处理。这样即使消息重复到达，也不会重复创建订单。

## 9. 什么情况下 Kafka 消费失败要重试？什么情况下不该重试？

参考回答：

像临时网络异常、数据库连接问题这类短时故障可以重试，因为有可能下一次就恢复了。像库存已经没有、数据本身非法、业务条件明确不成立这类情况就不该一直重试，不然只是浪费资源。这个区别我会结合异常分类来讲。

## 10. 你的死信队列是用来解决什么问题的？

参考回答：

死信队列主要处理“多次重试后还是失败”的消息，避免这些消息一直卡在主消费流程里。进入死信后，可以单独记录、排查或后续补偿。对于秒杀订单这种重要业务，死信队列可以提升问题可观察性。

## 11. 死信消息到来之后，你项目里后续怎么处理？

参考回答：

死信消息到了之后，我会把关键信息落到失败任务表或对应补偿表里，记录失败原因、重试次数和原始业务字段，后面再由补偿任务或人工排查处理。重点是不能让失败消息“消失”。

## 12. 如果 Kafka 暂时不可用，你的链路会发生什么？

参考回答：

如果 Kafka 暂时不可用，Relay 转发阶段会失败，Redis Stream 里的消息不会被成功 ACK，后面还可以继续重试处理。也就是说，入口不会立刻把数据完全丢掉，Stream 能起到缓冲作用。

## 13. 如果 Relay 成功从 Redis Stream 读到消息，但发 Kafka 失败了，会怎么处理？

参考回答：

这种情况下不能 ACK Redis Stream 的消息，否则消息就丢了。正确做法是让这条消息保留在 Pending 或未成功确认状态，后续继续重试发送 Kafka。也就是说，ACK 必须建立在“转发成功”之后。

## 14. 为什么消费者分组对秒杀场景重要？

参考回答：

因为消费者分组能让同一个 topic 的多个分区并行消费，提高吞吐量。秒杀场景下消息量大，如果没有消费组并行能力，单消费者容易成为瓶颈。消费组还能配合 rebalance 提高容错性。

## 15. 你为什么考虑过 consumer name 不能写死？

参考回答：

因为一旦多实例部署，如果 consumer name 写死，不同实例在 Redis Stream 或消费语义上可能会互相覆盖或混淆，尤其是 Pending List 恢复时更容易出问题。改成实例唯一之后，链路在多实例场景下会更安全。

## 16. 如果同一个用户的多条消息落到不同分区，会有什么问题？

参考回答：

最大的问题是顺序性可能丢掉。同一个用户的多次请求如果被不同消费者并行处理，就可能出现先后的业务顺序错乱。所以如果业务很依赖同用户顺序，通常会考虑用用户 id 作为 key，让相关消息尽量落到同一个分区。

## 17. 如果面试官问你 offset 是什么，你会怎么结合你的项目解释？

参考回答：

offset 可以理解成 Kafka 里每个分区消息的消费进度标记。消费者处理到哪里、确认到哪里，都要靠 offset 记录。结合我项目来说，只有订单消息真正处理成功后，才应该提交对应 offset。

## 18. 你项目里 Kafka 这块最容易被面试官追问的细节是什么？

参考回答：

我觉得最容易被追问的是“为什么既有 Redis Stream 又有 Kafka”、“手动 ACK 为什么重要”、“失败消息和死信怎么处理”这三块。因为这些问题最能看出你是不是真的理解消息链路，而不是只会堆技术名词。

---

## 八、分布式锁与一致性专题

---

### 1. 你项目里哪些地方用到了分布式锁？

参考回答：

最典型的是秒杀订单落库阶段，用来降低同一用户并发创建订单时的冲突。除此之外，在缓存击穿的互斥重建场景里，本质上也会涉及到分布式锁思路。

### 2. 你为什么选 Redisson，而不是自己手写一个简单 `setnx` 锁？

参考回答：

Redisson 更成熟，已经帮我处理了很多锁相关细节，比如续期、可重入、异常边界等。自己手写简单锁当然也能做，但容易在 owner 校验、过期、误删别人的锁这些细节上踩坑。我项目里后来也确实专门修过这类问题。

### 3. 你有没有自己实现过简单 Redis 锁？它和 Redisson 的差距在哪？

参考回答：

有过，简单版本通常就是 `setIfAbsent` 加过期时间，再在解锁时配合 Lua 校验 owner。它能覆盖基础场景，但和 Redisson 比起来，功能和健壮性都差很多，比如看门狗续期、可重入、封装程度都不如 Redisson。

## 4. 为什么解锁时不能直接 `delete key`?

参考回答:

因为锁有可能已经超时，或者当前线程根本不是持锁者。如果直接删 key，就有可能把别的线程刚拿到的锁删掉，造成严重并发问题。所以解锁一定要先确认这把锁是不是自己的。

## 5. 为什么安全解锁通常要“校验 owner 再删除”?

参考回答:

因为分布式锁的关键不只是“有没有锁”，而是“当前谁持有锁”。只有校验 owner 成功之后再删，才能避免误删别人持有的锁。这个校验通常还要保证原子性，所以常配合 Lua 做。

## 6. Lua 解锁脚本在这里起到了什么作用?

参考回答:

Lua 脚本可以把“先比较 owner，再删除 key”合成一个原子操作，避免中间被其他线程插进来。否则如果先 get 再 delete，中间就可能发生竞争条件。

## 7. 如果一个线程没拿到锁，却把别人的锁删掉了，会造成什么后果?

参考回答:

这样后果很严重，可能导致多个线程同时进入本该互斥的逻辑，比如同时重建缓存或者同时创建订单。表面看只是一个解锁小 bug，实际上会直接破坏并发保护。

## 8. 你是怎么发现并修复这个风险的?

参考回答：

我是从代码逻辑里发现这个风险的：如果没拿到锁的线程也执行了 finally 解锁，就存在误删别人锁的可能。修复思路就是让解锁只在真正持锁成功时发生，并且把锁值改成 owner token，再通过 Lua 原子解锁。

## 9. 如果锁过期时间设置太短，会有什么为题？

参考回答：

锁过期太短的话，业务还没执行完锁就没了，别的线程可能提前进来，导致本来需要互斥的逻辑失效。特别是高并发或耗时波动较大的场景，这个问题更明显。

## 10. 如果锁过期时间设置太长，又会有什么问题？

参考回答：

锁过期太长时，如果持锁线程异常退出，其他线程会等很久，系统吞吐量会下降，用户体验也会变差。所以锁时间要合理设置，既不能太短，也不能太长。

## 11. Redisson 的看门狗机制你了解吗？校招场景下你怎么简洁回答？

参考回答：

我会简单说：Redisson 有一个看门狗机制，会在业务线程还持有锁且正常运行时，自动帮你续期，避免锁因为时间太短提前过期。它的作用就是让锁更稳，不需要手动特别精细地估算业务执行时间。

## 12. 你的项目里哪些一致性问题“最终一致性”，哪些是“强一致性做不到也没必要”？

参考回答：

像本地缓存和 Redis 之间的同步，我接受的是最终一致性，因为通过广播失效和 TTL 兜底就足够了。像订单落库、库存扣减这种核心业务，我会更强调最终真实结果必须正确，所以要通过数据库约束、幂等和补偿去保证。不是所有地方都需要强一致，但关键链路一定要保证最终正确。

---

## 九、限流与稳定性专题

---

### 1. 你项目里为什么要做接口限流？

参考回答：

因为像发送验证码、登录、秒杀下单这类接口特别容易被高频调用，既可能被恶意刷，也可能在热门活动时把系统打爆。限流可以保护后端服务稳定，也是比较真实的工程需求。

### 2. 哪几个接口最需要限流？为什么？

参考回答：

我觉得最需要的是发送验证码接口、登录接口和秒杀下单接口。验证码和登录容易被恶意刷，秒杀接口则是天然高并发入口，对系统冲击最大。

### 3. 你的限流是基于什么实现的？

参考回答：

我是基于自定义注解、AOP 切面和 Redis 计数来做的。请求进来后，切面按某种维度生成限流 key，然后在 Redis 里做计数和过期控制，超过阈值就直接返回限流提示。

## 4. 自定义注解 `@RateLimit` 的作用是什么？

参考回答：

它的作用是把限流规则和具体接口绑定起来，让哪些接口要限流、按什么维度限流、阈值是多少这些信息更清晰。这样做比把限流逻辑写死在 controller 里更灵活。

## 5. 你是怎么区分“按 IP 限流”和“按用户限流”的？

参考回答：

本质上是限流 key 的组成不同。按 IP 限流时 key 里会带客户端 IP；按用户限流时 key 里会带用户 id。也就是说，限流逻辑本身差别不大，关键在于怎么生成唯一维度。

## 6. 你的限流算法更接近固定窗口、滑动窗口还是令牌桶？

参考回答：

我现在做的是比较轻量的固定窗口方案，用 Redis 做窗口内计数和过期时间控制。它实现简单，适合校招项目。后续如果想更平滑，可以再往滑动窗口或令牌桶升级。

## 7. 固定窗口限流有什么缺点？

参考回答：

最大的问题是窗口边界突刺，也就是一个窗口最后一秒和下一个窗口第一秒可能连续放过很多请求，实际限流效果不够平滑。所以它简单但不算最优。

## 8. 如果有人窗口边界疯狂请求，会发生什么？

参考回答：

可能会出现短时间内实际通过的请求数明显超过阈值，因为两个相邻窗口都各自放行了一部分请求。这也是固定窗口的典型问题。

## 9. 你项目里的限流返回值是怎么设计的？

参考回答：

一般会返回一个统一的失败结果，比如提示“请求过于频繁，请稍后再试”。对前端来说，最重要的是知道这是一个可理解的受控失败，而不是系统异常。

## 10. 如果面试官问你“后面要继续升级限流，你准备怎么改”，你怎么答？

参考回答：

我会说可以从三个方向升级：第一，把固定窗口改成滑动窗口或令牌桶；第二，把限流规则做成配置化甚至动态化；第三，在应用层之外再增加网关或 Nginx 层的粗粒度限流。这样从算法和部署层都能继续增强。

---

# 十、测试与压测专题

---

## 1. 你的项目里除了单元测试，还有哪些测试方式？

参考回答：

除了 Java 这边的单元或回归测试，我项目里还有 API 自动化测试、UI 自动化测试，以及一些压测和验证记录。也就是说，我不是只关注功能能不能写出来，也会关注怎么验证它。

## 2. 为什么你这个项目需要 API 自动化测试？

参考回答：

因为很多核心逻辑最终都是通过接口暴露出来的，比如登录、商户查询、秒杀下单。API 自动化测试可以比较快地验证主流程有没有被改坏，也适合做回归。

## 3. 为什么还要做 UI 自动化测试？

参考回答：

因为有些业务流程从页面角度更完整，比如登录、商户浏览、秒杀操作这些。UI 自动化虽然比 API 测试更重，但能补上接口测试覆盖不到的用户视角。

## 4. 你项目里哪些模块最值得补单元测试？

参考回答：

我觉得最值得补的是秒杀链路里的事务边界、缓存锁实现、消息消费幂等和配置规范这几块。因为这些地方一旦写错，问题通常不会体现在简单的接口成功失败上，而是会藏在并发和异常场景里。

## 5. 你最近补的测试主要覆盖了哪些风险点？

参考回答：

主要覆盖了三类风险：订单落库入口有没有事务边界、缓存锁是否会误删别人的锁、Relay consumer name 是否支持多实例唯一，以及示例配置里有没有保留不合适的演示口令。这些都比较偏工程风险，而不是简单业务 happy path。

## 6. 为什么“事务边界”也值得专门写测试？

参考回答：

因为事务边界一旦错了，最危险的问题是代码看起来能跑，但数据可能会不一致。比如库存扣了、订单没建成，这种问题非常难通过普通功能测试发现，所以我觉得值得专门验证。

## 7. 你为什么给配置模板也加了测试？

参考回答：

因为配置模板虽然不是业务代码，但它直接影响项目对外展示和团队使用方式。比如示例文件里保留了 `123456` 这种演示口令，会让仓库显得不够规范，所以我用一个很轻量的测试把这个规则固定下来。

## 8. 如果面试官问你“什么样的代码最难测”，你会结合项目里哪个模块来回答？

参考回答：

我会结合秒杀异步链路来答，因为它涉及 Redis、消息队列、数据库、锁和异常补偿，是跨组件的，不像纯工具类那么好测。这类代码最难的是场景复杂，不是写断言本身难。

## 9. 你做过压测吗？压测主要压的是哪条链路？

参考回答：

做过，主要是围绕秒杀链路和部分商户查询链路去看高并发下的表现。因为这两块一个是最典型的高并发写场景，一个是典型的高频读场景，比较有代表性。

## 10. 秒杀链路压测时，你最关注哪些指标？

参考回答：

我会重点看 QPS、平均响应时间、失败率、数据库压力、Redis 压力和消息堆积情况。对秒杀来说，不只是接口快不快，还要看后面的异步链路能不能接住流量。

## 11. 如果 QPS 上不去，你会优先排查哪几层？

参考回答：

我会先看入口是不是卡在 Redis 或 Lua，然后看 Kafka 或 Stream 有没有堆积，再看数据库是不是成为瓶颈，最后看应用线程池和连接池配置。基本就是按“入口拦截层、消息层、数据库层、应用资源层”这个顺序排查。

## 12. 如果面试官追问“你怎么证明自己真的压过”，你会拿哪些结果或现象来讲？

参考回答：

我会讲我关注的指标变化，比如请求量上来之后 Redis 压力先升、数据库压力相对更稳，或者消息积压什么时候开始出现，以及我做了哪些链路改造后这些现象有什么变化。这样回答比单说“我压过”更有说服力。

---

# 十一、项目优化与追问专题

---

## 1. 你项目里做过哪些“从能跑到更工程化”的优化？

参考回答：

主要有商户两级缓存、本地缓存失效广播、秒杀链路异步化、Kafka 死信和补偿、分布式锁风险修复、配置脱敏、测试补充这些。简单说就是从“功能做出来”往“并发、稳定性、可维护性”继续

补。

## 2. 你为什么把秒杀链路从简单实现改到现在这个版本？

参考回答：

因为简单实现虽然能跑，但在高并发下数据库压力太大，而且幂等、失败重试、补偿这些问题考虑得不够完整。我后来逐步把资格校验前移到 Redis，再通过消息队列做异步削峰，整体链路就更像真实业务系统。

## 3. 你自己最满意的一次改造是什么？具体解决了什么问题？

参考回答：

我最满意的是秒杀链路异步化和多层防线这次改造。它解决了数据库被高并发直接打爆的问题，也把一人一单、重复消费、落库失败补偿这些问题一起考虑进来了。这个改造前后，项目的工程感差别很明显。

## 4. 你在做改造时，有没有发现原方案看起来能用，但实际上有风险？举个例子。

参考回答：

有，最典型的是缓存锁实现。表面上看加锁、解锁都写了，但如果没拿到锁的线程也在 finally 里解锁，就可能删掉别人的锁。这种问题运行时不一定立刻暴露，但一旦高并发就很危险。

## 5. 如果让你再做一次这个项目，你会最先重构哪一块？

参考回答：

我会优先把文档、配置和消息链路抽象做得更统一一些，尤其是缓存和秒杀相关模块。因为业务能跑之后，真正影响后续维护效率的往往是结构清晰度，而不是少写一个接口。

## **6. 如果让你把这个项目部署成多实例线上版本，你最担心哪几个问题？**

参考回答：

我最担心的是本地缓存一致性、Redis 热点压力、消息消费的多实例协同、分布式锁边界和配置管理。单机和多机最大的区别就是很多原本简单的问题会变成一致性和协同问题。

## **7. 你觉得这个项目现在还有哪些不足？不要只说“可以继续优化”，请说具体点。**

参考回答：

具体不足有几个：限流算法还比较轻量，没做到更平滑的滑动窗口或令牌桶；监控和告警比较弱；集成测试还不够完整；另外部分文档和资料结构还可以继续整理。也就是说它已经有工程化意识，但离真正生产级还有距离。

## **8. 如果面试官问你“这个项目里哪块最像真实公司项目”，你会怎么答？**

参考回答：

我会答秒杀链路和缓存链路。因为这两块都有很明确的性能目标、并发问题和工程取舍，不只是“写接口”，而是要解决真实系统里常见的流量和一致性问题。

## **9. 如果面试官问你“哪块最不像真实公司项目”，你会怎么答？**

参考回答：

我会说监控、告警、灰度、权限体系这些部分还比较轻，不像公司项目那样完整。因为这个项目的重点是业务链路和核心工程点，不是完整地还原企业平台能力。

## 10. 如果让你只保留三个亮点写在简历上，你会选哪三个？

参考回答：

我会写三个：第一，基于 Redis Lua + Redis Stream + Kafka + MySQL 的秒杀异步链路；第二，基于 Caffeine + Redis + Pub/Sub 的两级缓存方案；第三，接口限流、失败补偿和回归测试这些稳定性增强。这样能同时体现并发、缓存和工程化能力。

---

## 十二、细节拷打专题

### 1. 你的秒杀 Lua 脚本入参具体有哪些？返回值有哪些？

参考回答：

核心入参一般包括优惠券 id、用户 id、订单 id 这几个关键字段。返回值我设计成状态码风格，比如 0 表示成功，1 表示库存不足，2 表示重复下单。这样 Java 侧判断逻辑会很清晰。

### 2. 你在 Redis 里给秒杀库存、订单集合分别用的 key 前缀是什么？

参考回答：

库存 key 一般是 `seckill:stock:{voucherId}`，一人一单集合一般是 `seckill:order:{voucherId}` 这种风格。前缀设计上我会尽量让 key 一眼能看出用途，便于排查。

### 3. 你的本地缓存和 Redis 缓存默认 TTL 大概是多少？

参考回答：

商户详情、本地缓存和 Redis 缓存的 TTL 我是区分开的，本地缓存一般更短，Redis 可以更长一些，商户类型列表还能再长一些。这里我会根据业务特点回答，而不是死背一个数值，因为不同对象 TTL 设计本身就不完全一样。

### 4. 你的 Kafka 消费者组名配置在哪里？

参考回答：

是在 `application.yml` 里配置的，通过 Spring Kafka 的消费者配置读取。这样比写死在代码里更规范，也方便后面调整。

### 5. 你的 Redis Stream 名字是什么？

参考回答：

秒杀订单这条链路的 Stream 名字是类似 `stream.orders` 这样的命名。它承担的是 Redis 入口消息流的角色。

### 6. 你的订单消息里为什么同时带 `orderId`、`userId`、`voucherId`？

参考回答：

因为消费者落库时需要这些最核心的业务字段。`orderId` 用来标识订单本身，`userId` 用于一人一单和分布式锁逻辑，`voucherId` 用于库存扣减和业务定位。缺任何一个都不方便做完整处理。

### 7. 你在代码里是怎么区分“重复下单”和“库存不足”的？

参考回答：

我是在 Lua 返回值和后端结果转换里区分的，不同状态码对应不同业务结果。这样接口返回和后续日志都能比较清楚地区分是哪一种失败。

## 8. 你的幂等主要靠哪几层一起完成？

参考回答：

主要靠 Redis 一人一单集合、数据库唯一索引、消费端重复插单兜底这几层一起做。不是只靠某一层单独保证，而是组合起来提升可靠性。

## 9. 你的 Redis 回滚库存脚本或逻辑，除了加库存，还做了什么？

参考回答：

除了把库存补回去，还要把一人一单集合里的用户状态移除掉，不然这个用户后续会一直被误判成已经下过单。也就是说，回滚不只是补库存，还要恢复业务状态。

## 10. 你的缓存更新时，为什么既要删 Redis，又要清本地缓存？

参考回答：

因为 Redis 和本地缓存都可能保存旧数据，只删其中一个是不够的。删 Redis 是为了清掉分布式共享缓存，清本地缓存是为了避免当前节点继续读旧值。再配合广播，才能尽量保证多节点一致。

## 11. 你的测试里为什么没有把所有东西都做集成测试？

参考回答：

因为集成测试覆盖范围大、依赖多、运行成本高，不适合所有细节都用它来验证。有些逻辑更适合轻量级单元或回归测试，比如配置模板校验、注解存在性、锁调用方式这些。测试也要分层，而不是一股脑全做集成。

## 12. 如果面试官把代码打开，随机指到一个类让你讲，你最怕他指到哪个类？为什么？

参考回答：

如果一定要说，我最不想被突然指到特别偏工具封装、而不是主业务链路的类，因为这种类细节很多，但业务成就感没那么强。不过即使被指到，我也会尽量先讲这个类解决了什么问题，再讲实现细节，而不是直接陷进代码行数里。

---

## 十三、模拟追问链

---

### 追问链 1：秒杀

#### 1. 你先讲一下秒杀链路。

参考回答：

我会先按“Redis Lua 资格校验和预扣 -> Redis Stream 入队 -> Relay 转发 Kafka -> Kafka 消费者异步落库 -> 失败重试或补偿”这个顺序讲。重点不是背组件名，而是讲清每一层为什么存在。

#### 2. 为什么不用数据库直接扣库存？

参考回答：

因为数据库抗不住秒杀瞬时高并发，而且大量无效请求没必要都打到数据库。把资格校验前移到 Redis，才能有效削峰。

#### 3. 为什么要 Lua？

参考回答：

因为要把库存判断、重复下单判断、预扣库存和写消息这些动作做成 Redis 内部原子操作。用 Lua 比在 Java 里发多次 Redis 命令更安全。

## 4. 为什么有了 Lua 还要数据库唯一索引？

参考回答：

因为 Redis 不是最终真相来源，异步链路里仍然可能出现重复消费或边界竞争。唯一索引是最终兜底。

## 5. 为什么有唯一索引还要 Redisson？

参考回答：

因为唯一索引只能在最后拦住错误，锁可以在更前面减少无效竞争。两者不是重复，而是位置不同的两道防线。

## 6. 如果 Kafka 消费失败怎么办？

参考回答：

先看异常是否可恢复，可恢复就重试；不可恢复就进死信或补偿处理。不能简单吞掉。

## 7. 如果 Redis 预扣成功但 MySQL 落库失败怎么办？

参考回答：

需要回滚 Redis 状态，至少把库存和一人一单集合恢复掉。否则会造成用户不能再次下单，库存也不准。

## 8. 你怎么证明你考虑过这些异常场景？

参考回答：

我会从代码层和测试层两个角度证明：代码里有异常分类、回滚和死信逻辑，测试里也补了事务边界和锁实现相关的回归验证。这样比单纯说“我考虑过”更可信。

## 追问链 2：缓存

### 1. 你讲一下商户缓存怎么做的。

参考回答：

商户详情采用两级缓存，先查本地 Caffeine，再查 Redis，最后才查数据库。更新时先更新数据库，再删 Redis、本地缓存，并用 Pub/Sub 广播失效。

### 2. 为什么要加 Caffeine？

参考回答：

因为热点数据如果每次都走 Redis，Redis 压力也会比较大。Caffeine 能进一步降低读取延迟和 Redis 压力。

### 3. 本地缓存一致性怎么保证？

参考回答：

主要靠更新时主动删除 + Pub/Sub 广播 + TTL 兜底。它不是强一致，但能做到比较可接受的最终一致。

### 4. Pub/Sub 丢消息怎么办？

参考回答：

丢消息时某些节点可能短时间读旧值，但 TTL 到期后会自动恢复。这个场景我接受最终一致，不追求强一致。

### 5. 逻辑过期和互斥锁你都用了，分别用在什么地方？

参考回答：

互斥锁更适合普通缓存重建；逻辑过期更适合热点数据快速返回旧值并异步重建。它们关注点不一样。

### 6. 你的锁如果写得不严谨，会出什么 bug？

参考回答：

最典型的的就是未持锁线程误删别人的锁，导致多个线程同时进入临界区。这个 bug 在并发场景下非常危险。

## 追问链 3：项目真实性

### 1. 这个项目里哪块是你自己改得最多的？

参考回答：

秒杀链路和缓存链路改得最多。因为这两块从业务正确性到性能优化都能展开很多。

### 2. 你改之前是什么样？

参考回答：

改之前更多是“功能能跑”的状态，像消息链路、补偿、两级缓存和锁安全这些工程细节没现在这么完整。

### 3. 你为什么想到这样改？

参考回答：

因为我在看链路时会先问自己：如果并发高了会怎样？如果消息失败了怎么办？如果缓存不一致了怎么办？很多改造就是从这些追问里出来的。

### 4. 你改完之后怎么验证的？

参考回答：

我会通过代码自查、局部回归测试、接口验证和压测结果去看是否真的改善。重点不是“我改了”，而是“我改完之后有没有证据证明更稳了”。

### 5. 你踩过什么坑？

参考回答：

比较典型的是锁释放逻辑、消息消费异常分类，还有缓存和数据库双写一致性这些坑。很多坑不是代码跑不起来，而是并发和异常场景下才暴露。

## 6. 最后你会怎么把这段经历写进简历？

参考回答：

我会强调自己负责后端整体开发，并突出两三个高价值点，比如秒杀异步链路、两级缓存和稳定性增强，而不是只写“完成了某某接口开发”。

---

## 十四、自测建议

### 1. 先只回答每个专题前 3 题，保证能顺下来。

参考回答：

这一步的目标不是讲满，而是先把主线说顺。特别是项目总览、缓存和秒杀，必须先做到“别人一问，我能顺着讲下来”。

### 2. 再补“细节拷打专题”，看自己哪里会卡住。

参考回答：

细节拷打最能检验你是不是真的做过。比如 key 命名、返回值设计、TTL、补偿逻辑这些，只要卡住就说明那块还得回头补。

### 3. 最后拿“模拟追问链”做完整演练。

参考回答：

模拟追问链的意义是让你从“会答单题”提升到“能经得住追问”。校招面试很常见的情况就是，第一问你答得还行，第二问第三问就乱了，所以这个阶段很重要。

如果你愿意，后面你可以按专题一段一段回答，我再继续扮演面试官追问你。